Module 8: A02 Security Misconfiguration & A03 Software Supply Chain Failures

OSINT Recon • ShopCo Flaw 2 Live Demo • Hardening Checklist • CVE Triage • Supply Chain Attack Taxonomy

Day 2 | OWASP Top 10:2025 | BBC Academy / JBI Training | Uses: vulnerable_shop.py, vulnerable_shop_fixed.py, OWASP Juice Shop, ZAP Proxy

Module Overview

A02 Security Misconfiguration asks: *is the environment around your code correctly hardened?* A03 Software Supply Chain Failures asks: *can you verify that what you installed is actually what the maintainer published?* Both categories are exploited **without ever touching your application code.** This module covers passive OSINT techniques attackers use to find misconfigured targets, a live demonstration of how a single misconfiguration flag grants remote code execution, a practical hardening checklist, and a progression from pip-audit CVE scanning to the broader supply chain attack taxonomy that defines A03:2025.

Part	OWASP	Activity	Time
A	A02	OSINT Reconnaissance — five sources attackers use before sending a single packet	20 mins
B	A02	ShopCo Flaw 2 Live Demo — DEBUG=True from misconfiguration to remote code execution	15 mins
C	A02	Flask Hardening Checklist — 17 items, ZAP verification, ShopCo & Juice Shop anchors	15 mins
D	A03	CVE Triage — CVSS vector decoding and the three-step triage process	15 mins
E	A03	Supply Chain Attack Taxonomy — beyond pip-audit to the five attack classes	15 mins

Part A — A02: OSINT Reconnaissance (20 minutes)

⚠ Legal & Ethical Requirement — Read Before Starting

All OSINT activities must be performed against: **(1) targets explicitly provided by your instructor,** (2) your own infrastructure, or **(3) publicly available fictional targets (example.com, iana.org).** Running these queries against BBC production infrastructure, third-party organisations, or any target without explicit written authorisation may constitute an offence under the **Computer Misuse Act 1990** and/or the **UK GDPR.** When in doubt, ask your instructor before running any query.

The attacker's workflow in two steps: (1) Run a Shodan search or Certificate Transparency lookup — **no packets sent to the target, fully passive, no legal risk.** (2) Cross-reference any version string or subdomain found with the NVD or Shodan Exploits. The entire reconnaissance phase for a targeted attack takes under 30 minutes and requires no specialist tools. This is why A02 is exploited at scale by automated scanners before human attackers ever engage.

	Shodan `shodan.io`	**DNS Recon** `dig, nslookup, dnsdumpster.com`	**Certificate Transparency** `crt.sh | censys.io`	**HTTP Response Headers** `ZAP Proxy | securityheaders.com`	**GitHub & Repos** `github.com | gitleaks | trufflehog`	**OWASP 2025**
What it reveals	• Open ports & service banners • TLS certificate details • Software version strings • Geographic location & ASN *Shodan indexes anything a TCP connection can read. The Exploits tab cross-references version strings with known PoC exploits.*	• Subdomains (staging, dev, admin, vpn) • Mail exchange records • SPF/DMARC policy gaps • Zone transfer misconfig • Internal IP ranges leaked via A records	• All TLS certs issued for a domain • Subdomains never intended to be public (*.internal, dev-, staging-) • Expired certs revealing historical infrastructure • Certificate mis-issuance events	• Server software & version (Server: header) • Framework version (X-Powered-By:) • Missing security headers (CSP, HSTS, X-Frame-Options) • Cookie flag deficiencies • CORS misconfiguration • Debug info in error responses	• Accidentally committed secrets (API keys, passwords, tokens) • Internal infrastructure details in config files • Historical secrets in git history (even after deletion) • Third-party service credentials	**A02** Security Misconfiguration **A03** Supply Chain (GitHub)

Worked queries	**Find Werkzeug debug consoles:**
	`http.title: "Werkzeug Debugger"`
	Find a version string:
	`product:"Apache httpd"`
		Enumerate subdomains:
		`dig +short ANY example.com`
		Check zone transfer:
		`dig axfr @ns1.example.com example.com`
			All certs for a domain:
			`https://crt.sh/?q=%.example.com&exclude=expired`
			Extract unique subdomains:
			`curl -s "https://crt.sh/?q=%.example.com&output=json"`
				Use ZAP to capture ShopCo's headers:
				Start ShopCo, set ZAP as proxy, browse to http://localhost:5000, then inspect the Response Headers tab in ZAP.
					Search GitHub for secrets:
					`"AKIA" extension:py "example.com" password filename:.env`	

	`version:"2.2"`		`| jq .[].name_value | sort -u`	**Check any live site:** `securityheaders.com` Aim for grade A or B.	**Scan git history locally:** `trufflehog git . --only-verified`	
What the attacker does next	"Server: Werkzeug/0.14.1" maps directly to CVE-2023-25577 (CVSS 7.5, DoS) and CVE-2019-14806 (debug PIN predictable → RCE). Shodan's Exploits tab cross-references automatically.	staging.internal.example.com resolving to a public IP reveals a staging environment — often with weaker credentials, test data, or debug mode enabled. Zone transfer success exposes the complete internal DNS map.	Certificate Transparency logs are public and permanent. staging-api.example.com with a cert reveals a target never hardened for public exposure. The attacker probes it for weaker authentication or debug interfaces.	"Server: Werkzeug/0.14.1" gives (1) exact version → check Shodan Exploits; (2) Python version → confirms aged deployment; (3) direct CVE mapping. Header suppression via Flask-Talisman costs one line of code.	A deleted AWS key (prefix AKIA…) committed and removed two years ago is still fully searchable on GitHub. Attacker finds it, tests against AWS STS:GetCallerIdentity, and enumerates IAM permissions in seconds.	

🔍 **ShopCo connection:** With ShopCo running and ZAP configured as your proxy, browse to **http://localhost:5000** and inspect the Response Headers tab in ZAP. Note the **Server:** header value. This is the exact string Shodan would index if ShopCo were internet-facing. How quickly could an attacker find the matching CVEs? Check on **nvd.nist.gov** or the Shodan Exploits tab.

Part B — A02: ShopCo Flaw 2 — DEBUG=True → Remote Code Execution (15 minutes)

What this demonstrates: A single misconfiguration flag — **DEBUG=True** — transforms a production Flask application into an interactive Python console accessible from any browser. No authentication required. This is an A02 Security Misconfiguration, not an injection vulnerability: the application code is not at fault. The deployment decision is.

The Vulnerable Code	Why It's Critical
The vulnerable configuration (vulnerable_shop.py, line 5):	**Why this is a Critical A02 Misconfiguration:** The Werkzeug debugger is designed for **local development only.** When enabled in any environment reachable by an attacker, any unhandled exception opens a full Python REPL in the browser. The REPL executes with the same OS privileges as the Flask process — typically the full permissions of the **ubuntu** user on your VM.

	Bandit detection: Run `bandit vulnerable_shop.py` and look for rule `B201: flask_debug_true` — rated **HIGH severity.** This is one of the five flaws Bandit catches automatically (compared with the three design failures it misses entirely — see Module 7).

```
app.config["DEBUG"] = True    # Flaw 2 — Werkzeug interactive debugger
                              #          enabled; any exception grants
                              #          browser-accessible Python REPL
                              #          → full OS command execution
```

Step-by-step demonstration

#	What to do	What you observe

1	**Ensure vulnerable_shop.py is running**
	`cd ~/shopco-security-lab`
	`python3 vulnerable_shop.py`
		You should see `* Debug mode: on` in the terminal. This confirms Flaw 2 is active.

2	**Trigger an unhandled exception in Firefox** Navigate to a route that does not exist: `http://localhost:5000/this-does-not-exist`	Instead of a plain 404 page, the **Werkzeug Interactive Debugger** appears, showing: • Full Python stack trace with file paths • Local variable values at every frame • The application's secret key in the session config • A ⌘ terminal icon next to every frame

3	**Open the interactive console**
	Click the ⌘ **terminal icon** on any stack frame.
	When prompted for a PIN, note that in Werkzeug ≤ 0.14.1 the PIN is algorithmically predictable from information already visible in the debugger (Flaw 2 / CVE-2019-14806).
	For this demonstration, the PIN has been pre-noted. Enter it.
		A live Python interpreter opens in the browser. You are now executing code on the server. Demonstrate:
		`>>> import os`
		`>>> os.popen('whoami').read()`
		`>>> os.popen('cat /etc/passwd').read()`
		Both commands execute on the VM and return results in the browser. **This is full remote code execution via a misconfiguration flag.**

4	**Run Bandit to show automated detection**
	Open a second terminal (Ctrl+Alt+T):
	`cd ~/shopco-security-lab`
	`pip3 install bandit`
	`bandit vulnerable_shop.py`
		Bandit reports:
		`>> Issue: [B201:flask_debug_true]`
		`Severity: HIGH  Confidence: HIGH`
		This is the CI gate that would have blocked this code from reaching production if Bandit were a required pipeline step.

5	**See the fix in vulnerable_shop_fixed.py** `grep -n DEBUG vulnerable_shop_fixed.py`	The fix reads the DEBUG flag from an environment variable: `app.config['DEBUG'] = os.environ.get('FLASK_DEBUG','false') == 'true'` DEBUG mode is now **off by default** unless explicitly set to true in the environment. It can never accidentally reach production via a committed source file.

Part C — A02: Flask Hardening Checklist (15 minutes)

📌 **Task:** Work through this checklist against your running ShopCo app. Use ZAP (with proxy configured) to verify each HTTP security header item. Tick each item that passes. For each that fails, note the ShopCo flaw or Juice Shop reference in the right column.

Section 1 — Application Configuration (A02)

✓	Item & fix	Vulnerable state	ZAP / browser verification

☐	**Secret key from environment — never hardcoded**
	`export FLASK_SECRET_KEY=$(openssl rand -hex 32)`
	`app.secret_key = os.environ['FLASK_SECRET_KEY']`
		ShopCo Flaw 1: app.secret_key = "password" — allows session cookie forgery by anyone who reads the source.	flask-unsign --decode --cookie "<paste cookie>" — if the secret is "password", session forgery is trivial.

☐	**Debug mode off in all non-local environments** `app.config['DEBUG'] = os.environ.get('FLASK_DEBUG','false') == 'true'`	ShopCo Flaw 2 (demonstrated in Part B): DEBUG=True → Werkzeug console → full RCE on any unhandled exception.	Trigger any 404. If Werkzeug debugger appears instead of a plain error page, debug mode is on.

☐	**Custom error handlers for 400, 403, 404, 500**
	`@app.errorhandler(500)`
	`def server_error(e):`
	`  app.logger.error(e, exc_info=True)`
	`  return jsonify({"error": "Internal error"}), 500`
	Full stack trace NEVER sent to browser.
		Juice Shop: unhandled errors expose framework version, file paths, and table names in stack traces — enabling targeted exploitation.	In ZAP: submit malformed input to any endpoint. Response body must not contain stack traces, file paths, or version strings.

☐	**No default or placeholder credentials**
	`# Audit before every deployment:`
	`grep -r "letmein|password|changeme" .`
	`trufflehog filesystem . --only-verified`
		ShopCo: admin/letmein is a demo credential. Default credentials are the first thing automated scanners try.	ZAP Forced Browse: try admin/admin, admin/password, admin/changeme against the login form.

Section 2 — HTTP Security Headers (A02) — all added via Flask-Talisman in one call

```
from flask_talisman import Talisman

Talisman(app,
  content_security_policy={'default-src': ['"self"'], 'script-src': ['"self"']},
  strict_transport_security=True,
  strict_transport_security_max_age=31536000,
  content_type_options=True,          # X-Content-Type-Options: nosniff
  frame_options='DENY',               # X-Frame-Options: DENY
  referrer_policy='strict-origin-when-cross-origin',
  permissions_policy={'geolocation': '()', 'microphone': '()', 'camera': '()'}
)
```

✓	Header	Risk if absent	ZAP check
☐	**Content-Security-Policy (CSP)**	No CSP: any stored or reflected XSS payload executes without restriction. ShopCo has no CSP. Juice Shop has a deliberately weak CSP for the XSS challenges.	ZAP Response Headers tab: Content-Security-Policy absent = fail. securityheaders.com grades headers automatically — aim for A.
☐	**Strict-Transport-Security (HSTS)**	Without HSTS, a man-in-the-middle can downgrade HTTPS to HTTP and intercept session cookies, even if the Secure flag is set — the first request over HTTP exposes the cookie.	Check Strict-Transport-Security in any HTTPS response. max-age must be ≥31536000 (1 year).
☐	**X-Content-Type-Options: nosniff**	Without nosniff, a browser may interpret an uploaded .txt file containing HTML as text/html and execute embedded scripts — a file upload XSS vector.	ZAP Response Headers tab: value must be exactly "nosniff". If absent, the app is vulnerable to MIME-type sniffing.
☐	**X-Frame-Options or frame-ancestors CSP**	Without X-Frame-Options, the app can be embedded in an attacker-controlled iframe, enabling clickjacking attacks that trick authenticated users into performing unintended actions.	Create a test page with <iframe src="http://localhost:5000"> and load it. If the iframe renders, clickjacking is possible.
☐	**Referrer-Policy**	Without Referrer-Policy, authenticated URLs with tokens, user IDs, or search terms in query parameters are sent to every third-party resource the page loads (analytics, CDN).	Navigate from an authenticated page to an external link. Check the Referer header in the outbound ZAP request — it must not contain session parameters.

⚡ **Quick wins:** Items 1 (secret key), 2 (debug mode), HSTS, and pip-audit in CI are each a one-line change with the highest risk-reduction-per-minute ratio on this checklist. If you do nothing else with this handout after the course, do these four first.

Part D — A03: CVE Triage — Reading CVSS Scores Properly (15 minutes)

📌 **Task:** Run pip-audit against **requirements_vulnerable.txt** in the ShopCo repo. For each HIGH or CRITICAL finding, look up the full CVSS vector on **nvd.nist.gov** and apply the three-step triage. The CVE Werkzeug 0.14.1 is the worked example.

```
pip-audit -r ~/shopco-security-lab/requirements_vulnerable.txt
```

CVSS Severity Bands — What Each Score Demands

Score	Severity	Required action	Real-world example from requirements_vulnerable.txt	Timescale	Context rule

9.0–10.0	**CRITICAL**	Patch or mitigate within 24–72 hours. If no patch: isolate the component, disable the feature, or take the service offline. Escalate immediately. Do not wait for a scheduled release cycle.	CVE-2019-14806 (Werkzeug 0.14.1): debug PIN predictable → RCE. CVE-2019-10906 (Jinja2 2.10): sandbox escape.	**24–72 hrs**	*AV:N/PR:N = treat as CRITICAL regardless of score band.*
7.0–8.9	**HIGH**	Patch within the next planned release cycle — typically 1–2 weeks for production systems. If in CISA KEV catalogue, treat as CRITICAL. Document the decision and residual risk.	CVE-2023-25577 (Werkzeug 0.14.1, CVSS 7.5): multipart form parsing → DoS. CVE-2018-18074 (requests 2.18.0): HTTPS credentials leaked to HTTP redirects.	**1–2 weeks**	*Check CISA KEV. If listed, escalate immediately.*
4.0–6.9	**MEDIUM**	Patch in the next regular maintenance window. Assess whether your deployment is actually exploitable using the CVSS vector analysis below. A MEDIUM score may be CRITICAL in your specific context.	CVE-2023-30861 (Flask 1.0.0, CVSS 7.5 — misclassified as Medium in some databases): session persistence after logout.	**Maintenance window**	*Read the vector, not just the score. Exploitability in your context overrides the band.*
0.1–3.9	**LOW**	Schedule for patching within the quarter. Acceptable to defer if the vulnerability requires	itsdangerous 0.24: predates timing-safe comparison improvements — theoretical timing attack on session tokens.	**Within quarter**	*Never silently defer. Document who accepted the risk and when it will*

		local access or user interaction your threat model makes unlikely. Document the risk acceptance decision with a named owner and review date.			*be reviewed.*

CVSS Vector Decoder — Worked Example: CVE-2023-25577 (Werkzeug 0.14.1)

The CVSS Vector

The full vector:

`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H`

Code	Means	Question to
`AV:N`	Network	*Is this compor internet-reach*
`AC:L`	Low complexity	*No special conditions — is repeatable*
`PR:N`	No privileges	*Unauthenticat attacker*
`UI:N`	No user interaction	*Server-side — click required*
`S:U`	Unchanged	*Impact limited Werkzeug pro*
`C:N`	No confidentiality impact	*No data disclo*
`I:N`	No integrity impact	*No data modif*
`A:H`	HIGH availability impact	*Service crash*

Applying the Three-Step Triage

Three-step triage for this CVE:

Step 1 — Assess exploitability:

- AV:N → network-exploitable. Is Werkzeug directly internet-accessible or behind a reverse proxy?
- AC:L / PR:N → no special conditions, no auth. HIGH urgency even behind a proxy.
- Does your application accept multipart form data? If not, the attack vector may not be reachable.

Step 2 — Check fix availability:

```
pip-audit -r requirements_vulnerable.txt
```

Output shows: fix available in Werkzeug ≥ 2.3.3. Check `pypi.org` changelog for the exact security release notes.

Step 3 — Decide and document:

- Patch: update to Werkzeug ≥ 2.3.3, record PR/ticket, date, and reviewer.
- Mitigate: if patching is blocked by a dependency conflict, add request size limits at the reverse proxy layer as a temporary compensating control. Document the expiry date for this decision.

> **"We know about it" is not risk acceptance.** Risk acceptance re named person with authority to accept that risk, a written rationale

Part E — A03: Supply Chain Attack Taxonomy — Beyond pip-audit (15 minutes)

> **What changed from A06:2021 to A03:2025:** A06:2021 "Vulnerable and Outdated Components" asked: *are your dependencies patched?* A03:2025 "Software Supply Chain Failures" asks: *has*

your supply chain been compromised? pip-audit answers the first question. The five attack types below represent threats that pip-audit alone would not detect.

The unifying principle: A supply chain attack exploits **trust.** You trust your package registry. Your registry trusts its maintainers. Your build pipeline trusts its runners. Attackers do not need to compromise your code — they compromise something you trust. The three defences are: **verify integrity** (hashes, signatures), **track provenance** (SBOM, SLSA attestations), and **minimise trust surface** (pin versions, pin SHAs, use OIDC, gate on CVE scans in CI).

Attack Type	Canonical Example	How It Works	Does pip-audit catch it?	Defence
Typosquatting *Registry Attack*	colourama (PyPI, 2017) — typo of colorama, installed a keylogger. Downloaded 500K times before removal.	Attacker publishes a malicious package with a name that is a common typo or visual lookalike. CI's pip install runs the malicious setup.py with full runner privileges.	Only if the package later gets a CVE. Not on day zero.	Hash-pinned requirements.txt. pip install --require-hashes rejects any package whose SHA-256 does not match, regardless of name.
Dependency Confusion *Namespace Hijacking*	Alex Birsan (2021): 35 companies including Apple, Microsoft, PayPal compromised. Internal package names uploaded to PyPI with version 99.0.0.	An internal package name is discovered in metadata. Attacker uploads a public package with the same name but higher version. pip prefers the public registry and installs the malicious package.	**No — the package is not known-vulnerable at the time of installation.**	Configure pip to use only your internal registry (index-url) or combine with hash pinning and no-index.
Compromised Legitimate Package *Maintainer Attack*	XZ Utils (CVE-2024-3094, CVSS 10.0): maintainer social-engineered over 2+ years. Backdoor in release tarball, not in git source.	A trusted maintainer is coerced or account-taken-over. The malicious code executes at install time, import time, or runtime — in the XZ case, only in systemd-linked builds, evading most test environments.	**Not on day zero. CVE published after discovery — organisations using it were already compromised.**	GPG signature verification. SBOM to track exact provenance. Monitor for unexpected new maintainers via deps.dev or socket.dev.
Build Pipeline Compromise *CI/CD Injection*	Codecov breach (2021): attacker modified the Codecov bash uploader script. Every CI pipeline running curl | bash exfiltrated cloud credentials.	Malicious PR injects commands into a workflow file, or a reusable Action is compromised when its tag is moved to a new commit. curl | bash in CI is the classic entry point.	**No — the attack is in the pipeline, not in a dependency.**	Pin GitHub Actions by full SHA not tag. Never curl | bash in CI. Use OIDC short-lived tokens, not stored secrets. SLSA provenance attestations.
Unsigned / Unverified Artifacts *Integrity Failure*	Linux Mint (2016): ISO image replaced with backdoored version. The official SHA-256 on the same compromised site was also updated.	pip install without --require-hashes installs whatever the registry returns. A compromised mirror or CDN can substitute a	Partially — pip-audit checks CVEs in installed packages but does not verify the integrity of the installed	pip install --require-hashes. Sigstore/pip verify (emerging standard). Publish hashes on a separate, trusted

		malicious wheel undetected.	artifact against the publisher's signature.	channel from the artifact.

SBOM — Why A03:2025 Requires It

Log4Shell (2021) affected tens of thousands of organisations that did not know they had log4j in their stack — embedded in transitive dependencies three or four levels deep. An SBOM would have made the affected surface area immediately queryable.

Generate an SBOM from your VM:

Detection tools on your VM

Tool	What it finds
`pip-audit`	Known CVEs in requirements (day-one detection of compromised packages only after CVE publication)
`cyclonedx-py`	Generates full SBOM — all direct and transitive deps with version and provenance
`trufflehog git`	Scans git history for accidentally committed secrets, even after deletion
`socket.dev / deps.dev`	Flags maintainer changes, new install scripts, obfuscated code in real-time — before CVEs exist

```
pip3 install cyclonedx-bom
cyclonedx-py environment -o ~/sbom.json

# Count total components (direct + transitive):
python3 -c "import json,sys; d=json.load(open('/root/sbom.json'));
print(len(d.get('components',[])))"

# How many did you install intentionally vs transitively?
# Find Werkzeug in the SBOM — is it a direct or transitive dependency?
grep -i "werkzeug" ~/sbom.json
```

Discussion & Reflection

Discussion Questions

For all delegates

1. The Shodan query `http.title:"Werkzeug Debugger"` returns real internet-facing systems with DEBUG=True enabled. Why do developers leave this in production, and what process controls would prevent it?

2. ShopCo Flaw 2 is a one-character difference from a secure configuration. At which point in a software delivery pipeline would it be cheapest to catch, and at which point is it most expensive?

3. Certificate Transparency logs are public and permanent. Every subdomain ever issued a certificate is discoverable. How should this

For developer delegates

4. pip-audit would not have detected the XZ Utils backdoor (CVE-2024-3094) before its publication. Given this, what is pip-audit actually useful for — and what does it give you false confidence about?

5. The Dependency Confusion attack exploits the fact that pip checks the public registry before internal registries. Describe the pip configuration change that would prevent this, and the trade-off it introduces.

For tester & sysadmin delegates

6. If you discovered that a production Werkzeug application had DEBUG=True

influence how your organisation names its internal services and staging environments?	enabled and was internet-reachable, what would your immediate response be before contacting the development team? What is the priority sequence? 7. A developer commits an AWS access key to a private GitHub repository and immediately deletes it. Why must you treat the credential as compromised regardless, and what are the required response steps?

Explorer *(breadth)*

E1: Visit securityheaders.com and grade the HTTP security headers for a public website of your choice (not BBC production). Identify which headers are missing and explain the risk each absence creates.

E2: Run `trufflehog git .` against the ShopCo repository. Does it detect the hardcoded secret key in Flaw 1? Note whether it finds it in the current HEAD or in a historical commit.

Practitioner *(depth)*

P1: Apply the complete three-step triage to every HIGH and CRITICAL finding from `pip-audit -r requirements_vulnerable.txt`. For each: read the CVSS vector, assess exploitability in a typical Flask deployment behind nginx, and write a one-sentence risk acceptance or patch decision with a named owner.

P2: Generate an SBOM for the ShopCo fixed app environment and count the transitive dependencies. For each package flagged by pip-audit, determine whether it is a direct or transitive dependency — how does this affect patch priority?

Expert *(synthesis)*

X1: Design a complete CI/CD security pipeline for ShopCo using GitHub Actions that implements: Bandit SAST on every PR, pip-audit gating on HIGH/CRITICAL CVEs, hash-pinned requirements.txt, OIDC-based AWS credentials, and Actions pinned by SHA. Write the complete `.github/workflows/security.yml` file.

X2: The XZ Utils backdoor was hidden in the **release tarball** but not in the **git source.** Explain how SLSA provenance attestations and SBOM verification would — or would not — have detected this attack earlier, and what the residual risk is even with these controls in place.

The connection between A02 and A03

Both categories are about the **environment around your code** rather than the code itself. A developer can write perfect application logic and still be compromised through a misconfigured debug flag (A02) or a malicious upstream package (A03). The old question — “is this version patched?” — is necessary but no longer sufficient. The new question is: “can I verify that what I installed is what the maintainer published, and is my environment configured so that no single misconfiguration creates a path to remote code execution?”